

PayPal TX Ledger Guide (Codebase-Verified, Chronological, Ledger-Focused)

This guide is based on your current codebase structure and existing PayPal checkout flow. It is a blueprint + implementation templates document (not yet applied runtime code).

Date verified against codebase: 2026-02-24

Table of Contents

1. Scope and Ground Rules
2. Current Flow (Verified from Code)
3. What We Are Changing (and Not Changing)
4. Final Target Flow (End-to-End)
5. Implementation Order (Chronological)
6. Environment Variables
7. Prisma Schema (PayPal Ledger) - Complete and Typed
8. Prisma Commands (No Separate Prisma Config File)
9. Prisma Client Wrapper
10. Status Constants and Vocabulary (string-based, enum deferred)
11. Ledger Types (JSON payload typing strategy)
12. Checkout Intent Store (client)
13. PayPal Intent Endpoint (server)
14. PayPalCheckoutChildren Integration (replace createOrderAction path)
15. Authorize Route Update (ledger write)
16. Capture Route Update (ledger write)
17. Approve Hook Update (`usePayPalTXApproveCallback`)
18. Order Status Endpoint
19. Polling + UI Mapping (Processing Modal / Confirmation Page)
20. Webhook Route Upgrade (idempotent + post-processing trigger)
21. Server Post-Processing Runner (ledger-driven)
22. Fulfillment Provider Adapter (Merchize isolated)
23. Admin Auth (header secret)
24. Admin Ledger UI + Retry Actions
25. Sandbox/Dev Fallback Trigger
26. Rollout and Migration Order (safe deployment)

- 27. Package Scripts (no extra Prisma config file)
 - 28. Testing Matrix (local/sandbox/live)
 - 29. Deferred Known Issues in Current Code (tracked)
 - 30. Decision Defaults
 - 31. Optional Overrides (only if rejecting defaults)
 - 32. Future: Real-Time Notifications at Scale (SSE + Pub/Sub)
-

1) Scope and Ground Rules

Goal

Move post-payment processing (receipt generation/upload, payment save, fulfillment push) out of the browser and into a durable server-side ledger + webhook flow.

Non-goals

- Rewriting your entire checkout UI.
- Replacing your existing PayPal order creation route.
- Replacing your OTP flow.

Ground rules for this guide

- Use your existing route naming convention (`/next-api/...`).
 - Keep Merchize SQLite setup untouched.
 - Do **not** require a separate Prisma config file for the PayPal ledger schema.
 - Keep status fields as strings for now (Prisma enum deferred).
 - Isolate fulfillment-provider details behind a ledger runner adapter boundary.
-

2) Current Flow (Verified from Code)

2.1 Checkout + OTP

- Checkout info form lives in:
 - `src/components/UI/Shop/Checkout/UserCheckoutSummary/BasicCheckoutInfo.tsx`
- Django order-intent OTP send/verify flow lives in:
 - `src/lib/hooks/shopHooks/checkout/useDjangoOrderIntentEmailVerification.ts`

- `src/lib/hooks/shopHooks/checkout/djangoOrderIntentMutationHooks.ts`
- Verified Django order-intent identifiers are stored in:
 - `src/stores/shop_stores/checkoutStore/djangoOrderIntentStore.ts`

2.2 PayPal order creation

- `src/components/UI/Shop/Checkout/Paypal/PayPalCheckoutChildren.tsx` calls `createOrderAction(...)`.
- `src/actions/shop/paypal/createOrderAction.ts` decrypts payload and calls:
 - `/next-api/paypal/orders/create-order`
- `src/app/api/paypal/orders/create-order/route.ts` validates pricing via `getOrderFinalDetails(...)` and creates the PayPal order.

2.3 Approve, authorize, capture

- `src/lib/hooks/shopHooks/checkout/usePayPalTXApproveCallback.ts` calls:
 - `/next-api/paypal/orders/authorize`
 - `/next-api/paypal/orders/capture`
- Routes:
 - `src/app/api/paypal/orders/authorize/route.ts`
 - `src/app/api/paypal/orders/capture/route.ts`

2.4 Post-processing (currently client-oriented but partially disabled)

The intended client path is:

- `savePaymentReceiptToCloud`
- `savePaymentDataToBackend`
- `sendMerchizeOrderDetailsToBackend`

Current state:

- `usePayPalTXApproveCallback.ts` starts processing UI but post-processing call is commented out.
- This can leave the modal active without progress completion.

2.5 Webhook

- `src/app/api/paypal/webhooks/ledger-transaction-events/route.ts` already includes:
 - raw-body parsing pattern (`req.text()`)
 - optional signature verification
 - Business processing in webhook is still placeholder comments.
-

3) What We Are Changing (and Not Changing)

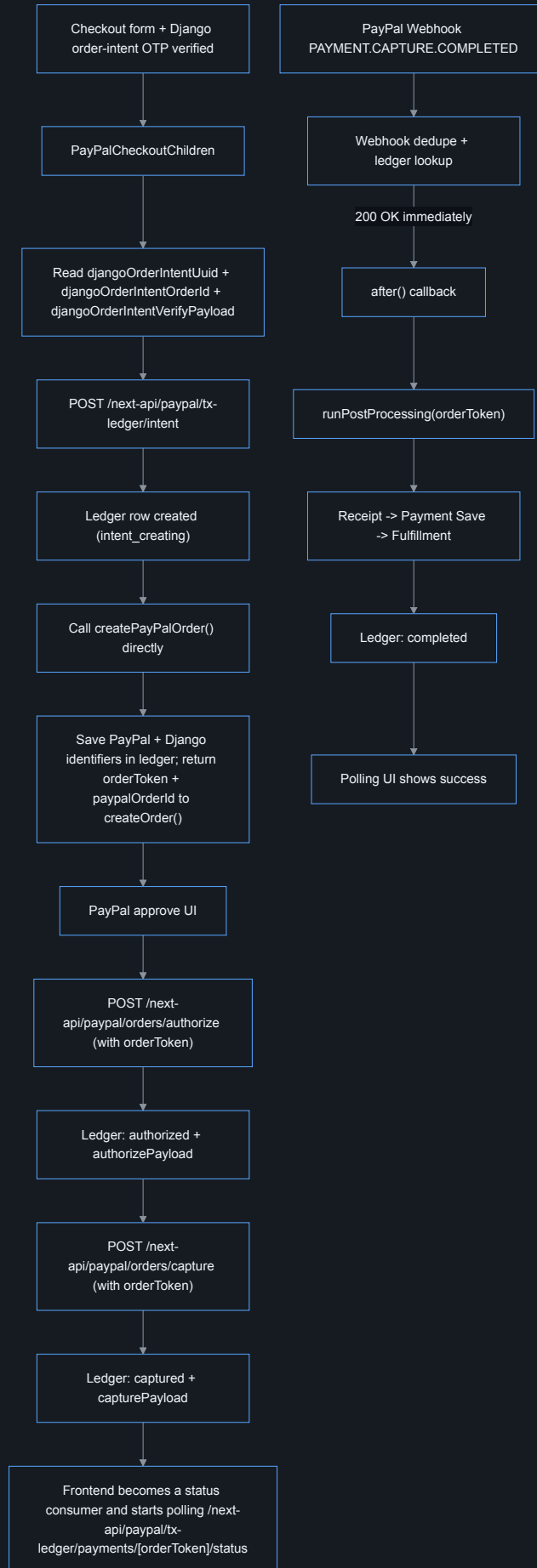
We are changing

- Add PayPal ledger database tables (Neon/Postgres via Prisma).
- Extract create-order business logic into `src/lib/paypal/createPayPalOrder.ts` (shared function).
- Refactor `create-order` route to be a thin wrapper around the shared function.
- Add a new `/next-api/paypal/tx-ledger/intent` route that calls the shared function directly (no self-referential HTTP fetch).
- Write ledger state during authorize/capture.
- Make webhook idempotent and trigger server post-processing.
- Add `/next-api/paypal/tx-ledger/payments/[orderToken]/status` polling route.
- Add admin ledger retry UI/actions.

We are not changing

- Existing `/next-api/paypal/orders/create-order` route behavior (same inputs/outputs, logic extracted to shared function).
 - Existing `/next-api/paypal/orders/authorize` and `/capture` endpoints as the ownership point for authorize/capture.
 - OTP verification backend contract.
 - Merchize SQLite catalog DB and its Prisma config.
-

4) Final Target Flow (End-to-End)



5) Implementation Order (Chronological)

Implement in this order to avoid half-migrated behavior:

1. Add ledger schema + generate client.
2. Add status constants/types.
3. Add ledger client wrapper.
4. Add intent store (client).
5. Extract create-order logic into `src/lib/paypal/createPayPalOrder.ts` and refactor `create-order` route to use it.
6. Add `/next-api/paypal/tx-ledger/intent` (calls `createPayPalOrder` directly, no self-referential fetch).
7. Update `PayPalCheckoutChildren` to use intent route.
8. Update authorize route to persist ledger.
9. Update capture route to persist ledger.
10. Update approve hook to pass `orderToken`, remove client post-processing, and redirect into the confirmation/status flow.
11. Add `/next-api/paypal/tx-ledger/payments/[orderToken]/status`.
12. Add polling + UI mapping.
13. Upgrade webhook (idempotency + ledger updates + post-processing trigger).
14. Add server post-processing runner + fulfillment adapter.
15. Add admin auth + admin UI/actions.
16. Add dev fallback trigger endpoint.
17. Retire client-side post-processing from checkout runtime.

Important transition note:

- After Step 10, the old browser-owned completion path is intentionally gone.
- Step 11 and Step 12 restore the frontend read path.
- Step 13 and Step 14 restore durable backend completion.
- So if you adopt the thin approve hook early, do not expect receipt upload / payment save / fulfillment to run again until Steps 13 and 14 are implemented.

6) Environment Variables

env

```
1 # Existing (already in project)
2 DATABASE_URL="file:./dev.db"
3 PAYPAL_WEBHOOK_VERIFY="true"
4 PAYPAL_ENV="sandbox"
5 NEXT_PUBLIC_PAYPAL_ENV="sandbox"
```

```

6
7 # Preferred PayPal browser vars
8 NEXT_PUBLIC_PAYPAL_CLIENT_ID_SANDBOX="..."
9 NEXT_PUBLIC_PAYPAL_CLIENT_ID_LIVE="..."
10
11 # Preferred PayPal server vars
12 PAYPAL_CLIENT_ID_SANDBOX="..."
13 PAYPAL_CLIENT_ID_LIVE="..."
14 PAYPAL_CLIENT_SECRET_SANDBOX="..."
15 PAYPAL_CLIENT_SECRET_LIVE="..."
16 PAYPAL_WEBHOOK_ID_SANDBOX="..."
17 PAYPAL_WEBHOOK_ID_LIVE="..."
18
19 # Backward-compatible fallback vars (supported, but no longer preferred)
20 NEXT_PUBLIC_PAYPAL_CLIENT_ID="..."
21 PAYPAL_CLIENT_ID="..."
22 PAYPAL_CLIENT_SECRET="..."
23 PAYPAL_WEBHOOK_ID="..."
24
25 # Existing (OTP / backend integrations - keep as-is)
26 NEXT_PUBLIC_BASE_URL="..."
27 NEXT_PUBLIC_SHOP_CHECKOUT_OTP_VERIFICATION_API_KEY="..." # existing; security issue tracked separately
28
29 # New (ledger)
30 PAYPAL_TX_LEDGER_NEON_DB_STRING="postgresql://USER:PASSWORD@HOST/DB?sslmode=require"
31 ADMIN_SECRET_KEY="long-random-secret"
32 SHOP_SERVER_ACTIONS_CRYPT0_SECRET="long-random-secret"

```

Notes:

- Browser PayPal config is resolved from:
 - `src/lib/paypal/publicPayPalConfig.ts`
- Server PayPal config is resolved from:
 - `src/lib/paypal/serverPayPalConfig.ts`
- `NEXT_PUBLIC_PAYPAL_CLIENT_ID_*` values are public by design. They are not secrets.
- `PAYPAL_CLIENT_SECRET_*` and `PAYPAL_WEBHOOK_ID_*` stay server-only.
- The current code supports both the new split variables and the old single-name fallbacks during migration.
- For clean deployments, keep `PAYPAL_ENV` and `NEXT_PUBLIC_PAYPAL_ENV` aligned.
- Merchize SQLite continues to use your existing `prisma.config.ts`.
- PayPal ledger schema also gets its datasource URL from `prisma.config.ts` in Prisma 7.
- The root `prisma.config.ts` should switch datasource URLs based on the `--schema` target.
- Do **not** reuse `NEXT_PUBLIC_SHOP_CHECKOUT_OTP_VERIFICATION_API_KEY` for webhook/server post-processing. Keep a separate server-only secret for those actions.
- This guide intentionally avoids requiring a second Prisma config file for the ledger.

7) Prisma Schema (PayPal Ledger) - Complete and Typed

File:

- `prisma/shop/paypal/paypalTXLedger.schema.prisma`

This schema is ledger-focused and keeps fulfillment-provider specifics isolated from the schema itself.

prisma

```
1  datasource db {
2    provider = "postgresql"
3    // URL is set in prisma.config.ts (Prisma 7 style)
4  }
5
6  generator paypalTxLedger {
7    provider = "prisma-client"
8    output   = "../../src/lib/prisma/shop/paypal/txLedger/generated/paypalTxLedger"
9  }
10
11  model PaypalIntent {
12    id                String    @id @default(cuid())
13    orderToken        String    @unique
14    paypalOrderId      String?   @unique
15    paypalAuthorizationId String?
16    djangoOrderIntentUuid String?
17    djangoOrderIntentOrderId String?
18    customerName      String
19    customerEmail     String
20    userId            String?
21    countryIso2       String?
22    countryIso3       String?
23    initialCurrency   String?
24    cartSnapshot      Json
25    shippingSnapshot  Json
26    // Raw payloads needed to reuse your existing server actions
27    authorizePayload   Json?
28    capturePayload     Json?
29    djangoOrderIntentPayload Json?
30    djangoOrderIntentVerifyPayload Json?
31    djangoPaymentSaveResponsePayload Json?
32    merchizeFulfillmentRequestPayload Json?
33    merchizeFulfillmentResponsePayload Json?
34    merchizeFulfillmentProcessingId String?
35    merchizeProviderOrderId String?
36    merchizeProviderOrderCode String?
37    fulfillmentAddressOverride Json?
38    fulfillmentAddressOverrideReason String?
39    fulfillmentAddressOverriddenBy String?
40    fulfillmentAddressOverriddenAt DateTime?
41    djangoPaymentSaveCustomId String?
42    receiptLink        String?
43    receiptFile        String?
44    status             String
45    lastEventType      String?
46    lastErrorCode      String?
47    lastErrorMessage   String?
48    retryCount         Int       @default(0)
49    postProcessingLockId String?
50    postProcessingLockedAt DateTime?
51    postProcessingLockExpiresAt DateTime?
52    processingCompletedAt DateTime?
53    createdAt          DateTime @default(now())
54    updatedAt          DateTime @updatedAt
55
56    @@index([customerEmail])
57    @@index([djangoOrderIntentUuid])
58    @@index([djangoOrderIntentOrderId])
59    @@index([djangoPaymentSaveCustomId])
60    @@index([status])
61    @@index([createdAt])
62    @@index([postProcessingLockExpiresAt])
63  }
64
```



```

65 model PaypalWebhookEvent {
66   id          String    @id @default(cuid())
67   eventId     String    @unique
68   eventType   String
69   paypalOrderId String?
70   payload     Json
71   processingStatus String @default("received")
72   attemptCount Int       @default(0)
73   lastAttemptAt DateTime?
74   processedAt  DateTime?
75   lastErrorMessage String?
76   createdAt    DateTime @default(now())
77
78   @@index([paypalOrderId])
79   @@index([eventType])
80   @@index([processingStatus])
81   @@index([processedAt])
82   @@index([createdAt])
83 }

```

Why no Prisma relation between `PaypalIntent` and `PaypalWebhookEvent`?

- PayPal webhook events may arrive before a ledger row is fully correlated.
- Event dedupe only needs `eventId`.
- A plain indexed `paypalOrderId` field is simpler and avoids relational coupling at this stage. A formal FK can be added when the schema stabilizes.

Why the extra tracking fields matter:

- `PaypalWebhookEvent.processedAt` prevents false "already handled" acks when a prior attempt failed.
- `PaypalIntent.postProcessingLock*` gives `runPostProcessing(...)` a DB-backed lease so only one worker can own side effects at a time.

8) Prisma Commands (No Separate Prisma Config File)

Because you do **not** want a separate Prisma config file for the ledger:

- Keep `prisma.config.ts` for Merchize SQLite.
- Extend the root `prisma.config.ts` so it selects the correct datasource URL based on `--schema`.
- Require `--schema` explicitly for every Prisma command. Do not let the config guess.
- For PayPal ledger, run Prisma commands explicitly with `--schema`.

Generate PayPal ledger client

```
bash
```

```
1 yarn prisma generate --schema prisma/shop/paypal/paypalTXLedger.schema.prisma
```

Create migration (development)

bash

```
1 yarn prisma migrate dev \  
2   --schema prisma/shop/paypal/paypalTXLedger.schema.prisma \  
3   --name init_paypal_tx_ledger
```

Deploy migrations (production)

bash

```
1 yarn prisma migrate deploy --schema prisma/shop/paypal/paypalTXLedger.schema.prisma
```

Inspect / validate

bash

```
1 yarn prisma validate --schema prisma/shop/paypal/paypalTXLedger.schema.prisma  
2 yarn prisma format --schema prisma/shop/paypal/paypalTXLedger.schema.prisma
```

This avoids conflicts with:

- `prisma.config.ts` (Merchize / SQLite)

9) Prisma Client Wrapper

File:

- `src/lib/prisma/shop/paypal/paypalTxLedger.ts`

ts

```
1 import { PrismaClient } from '@lib/prisma/shop/paypal/txLedger/generated/paypalTxLedger/client';  
2  
3 declare global {  
4   // eslint-disable-next-line no-var  
5   var __paypalTxLedger__: PrismaClient | undefined;  
6 }  
7  
8 export const paypalTxLedger = global.__paypalTxLedger__ ?? new PrismaClient();  
9  
10 if (process.env.NODE_ENV !== 'production') {  
11   global.__paypalTxLedger__ = paypalTxLedger;  
12 }
```

Why this wrapper:

- Prevents connection churn in dev/hot reload.
 - Gives one import path used by routes/hooks/server functions.
-

10) Status Constants and Vocabulary (string-based, enum deferred)

Decision default is string statuses (centralized constants), with Prisma enum conversion deferred.

File:

- `src/lib/paypal/txLedger/status.ts`

```
ts
1  export const PAYPAL_LEDGER_STATUS = {
2    INTENT_CREATING: 'intent_creating',
3    INTENT_CREATED: 'intent_created',
4    AUTHORIZED: 'authorized',
5    CAPTURED: 'captured',
6    RECEIPT_UPLOADED: 'receipt_uploaded',
7    PAYMENT_SAVED: 'payment_saved',
8    COMPLETED: 'completed',
9    PENDING: 'pending',
10   REFUNDED: 'refunded',
11   ERROR: 'error',
12 } as const;
13
14 export type PayPalLedgerStatus =
15   (typeof PAYPAL_LEDGER_STATUS)[keyof typeof PAYPAL_LEDGER_STATUS];
```

Why this matters:

- Prevents typos while keeping Prisma schema flexible.
-

11) Ledger Types (JSON payload typing strategy)

Store raw JSON in DB, but type it at read boundaries.

File:

- `src/lib/paypal/txLedger/types.ts`

```
ts
1  import type { OrderResponseBody } from '@paypal/paypal-js';
2  import type { OrdersCapture } from '@paypal/paypal-server-sdk';
3
4  export type LedgerAuthorizePayload = OrderResponseBody;
5  export type LedgerCapturePayload = OrdersCapture;
6
```

```
7   export type LedgerCartSnapshotItem = {
8     variantId: string;
9     quantity: number;
10    title?: string;
11    slug?: string;
12    itemDetail?: unknown;
13  };
```

Policy:

- Store raw responses exactly as returned by existing routes.
 - Normalize only when needed by post-processing or UI.
-

12) Checkout Intent Store (client)

File:

- `src/stores/shop_stores/checkoutStore/paypalIntentStore.ts`

```
ts

1  'use client';
2  import { create } from 'zustand';
3
4  type PayPalIntentState = {
5    orderToken?: string;
6    setIntent: (payload: { orderToken: string }) => void;
7    clearIntent: () => void;
8  };
9
10 export const usePayPalIntentStore = create<PayPalIntentState>((set) => ({
11   orderToken: undefined,
12   setIntent: ({ orderToken }) => set({ orderToken }),
13   clearIntent: () => set({ orderToken: undefined }),
14 }));
```

This is intentionally small and separate from your existing checkout store.

Design note:

- Keep only `orderToken` in browser app state.
 - Keep `paypalOrderId` as a server-side ledger correlation value.
 - The browser still briefly receives `paypalOrderId` because PayPal's SDK requires `createOrder()` to return it, but your own client state should not retain it after that callback returns.
-

13) PayPal Intent Endpoint (server)

13a) Extract create-order logic into a shared function

The existing `create-order` route handler contains the full order-creation business logic inline. Before adding the intent endpoint, extract that logic into a directly-importable function so the intent route can call it without an HTTP round-trip to itself.

Why this matters:

- A self-referential `fetch()` (route calling its own server) can deadlock under load when all Node.js workers are occupied waiting on themselves.
- Direct function calls are faster, testable, and do not depend on `host` header availability.

New file:

- `src/lib/paypal/createPayPalOrder.ts`

ts

```
1  import {
2    OrdersController,
3    CheckoutPaymentIntent,
4    OrderRequest,
5    ItemCategory,
6    Item,
7  } from '@paypal/paypal-server-sdk';
8  import { paypalClient } from '@lib/paymentClients/paypalClient';
9  import { getOrderFinalDetails } from '@actions/shop/checkout/getOrderFinalDetails';
10 import { PAYPAL_CURRENCY_CODES } from '@datasets/shop_general/paypal_currency_specifics';
11 import { removeOrKeepDecimalPrecision } from '@actions/merchize/getMerchizeTotalWithShipping';
12 import { randomUUID } from 'crypto';
13 import { format } from 'date-fns';
14
15 const orders = new OrdersController(paypalClient);
16
17 const shippingPreferenceForPaymentContext = {
18   experienceContext: {
19     shippingPreference: 'SET_PROVIDED_ADDRESS',
20     brandName: 'Codex Christi',
21   },
22 };
23
24 export type CreatePayPalOrderArgs = {
25   cart: Array<{ title: string; itemDetail: any; quantity: number; [k: string]: any }>;
26   customer: { name: string; email: string };
27   country: string;
28   country_iso_3?: string;
29   initialCurrency?: string;
30   delivery_address: {
31     shipping_address_line_1: string;
32     shipping_address_line_2?: string;
33     shipping_state?: string;
34     shipping_city?: string;
35     zip_code?: string;
36     [k: string]: any;
37   };
38 };
39
40 /**
```



```
45 export async function createPayPalOrder(args: CreatePayPalOrderArgs) {
46   const { cart, customer, country, country_iso_3, initialCurrency, delivery_address } = args;
47
48   if (!cart || cart.length === 0) throw new Error('Missing cart');
49   if (!customer) throw new Error('Missing customer');
50
51   const paypalSupportsCurrency = PAYPAL_CURRENCY_CODES.includes(
52     initialCurrency as (typeof PAYPAL_CURRENCY_CODES)[number],
53   );
54
55   const orderDetailsFromServer = await getOrderFinalDetails(
56     cart,
57     country_iso_3 ?? 'USA',
58     'merchize',
59   );
60   const { finalPricesWithShippingFee } = orderDetailsFromServer || {};
61   const { currency, retailPriceTotalNum, shippingPriceNum, multiplier } =
62     finalPricesWithShippingFee || {};
63
64   const currencyCode = currency && paypalSupportsCurrency ? currency : 'USD';
65   const getAdjAmount = (num: number) => (paypalSupportsCurrency ? num : num / (multiplier ?? 1));
66   const adjShipping = getAdjAmount(shippingPriceNum!);
67
68   const cartItemsForPaypalBodyContext = cart.map(
69     async ({ title, itemDetail, quantity }) =>
70     ({
71       name: title,
72       unitAmount: {
73         currencyCode: currency,
74         value: String(
75           await removeOrKeepDecimalPrecision(
76             currency!,
77             getAdjAmount(itemDetail.retail_price * (multiplier ?? 1)),
78           ),
79       ),
80     },
81     quantity: String(quantity),
82     description: title,
83     sku: itemDetail.sku_seller,
84     url: `https://codexchristi.shop/product/${itemDetail.product}`,
```

```

85     category: ItemCategory.PhysicalGoods,
86   }) as Item,
87 );
88
89 if (!retailPriceTotalNum || !shippingPriceNum) {
90   throw new Error('Invalid Price!! Aborting...');
91 }
92
93 const resolvedCartItems = await Promise.all(cartItemsForPaypalBodyContext);
94 const itemsContextTotal = await removeOrKeepDecimalPrecision(
95   currencyCode,
96   resolvedCartItems.reduce(
97     (accum, itm) => accum + Number(itm.unitAmount.value) * Number(itm.quantity),
98     0,
99   ),
100 );
101
102 const payload = {
103   body: {
104     intent: 'AUTHORIZE' as CheckoutPaymentIntent,
105     purchaseUnits: [
106       {
107         description: `Codex Christi Shop Order for ${customer.name} on ${format(
108           new Date(Date.now()),
109           "EEEE d 'of' MMMM yyyy hh:mm a",
110         )}
111       },
112       {
113         amount: {
114           currencyCode,
115           value: String(
116             await removeOrKeepDecimalPrecision(currencyCode, itemsContextTotal + adjShipping),
117           ),
118           breakdown: {
119             itemTotal: { currencyCode, value: String(itemsContextTotal) },
120             shipping: {
121               currencyCode,
122               value: String(await removeOrKeepDecimalPrecision(currencyCode, adjShipping)),
123             },
124           },
125           shipping: {
126             name: { fullName: customer.name },
127             address: {
128               addressLine1: delivery_address.shipping_address_line_1,
129               addressLine2: delivery_address.shipping_address_line_2,
130               adminArea1: delivery_address.shipping_state,
131               adminArea2: delivery_address.shipping_city,
132               postalCode: delivery_address.zip_code,
133               countryCode: country,
134             },
135             emailAddress: customer.email,
136           },
137             customId: randomUUID() ?? customer?.email ?? '',
138             items: paypalSupportsCurrency ? resolvedCartItems : undefined,
139           },
140         payer: {
141           name: { givenName: customer.name },
142           emailAddress: customer.email,
143         },
144         paymentSource: {
145           paypal: shippingPreferenceForPaymentContext,
146           card: {
147             experienceContext: shippingPreferenceForPaymentContext.experienceContext,
148             attributes: { verification: { method: 'SCA_WHEN_REQUIRED' } },
149           },
150           venmo: shippingPreferenceForPaymentContext,
151         },

```



```

152     } as OrderRequest,
153     prefer: 'return=representation',
154   };
155
156   const { body: resBody } = await orders.createOrder(payload);
157   return resBody;
158 }

```

Then update the existing create-order route to use it:

Updated file:

- `src/app/api/paypal/orders/create-order/route.ts`

```

ts
1  import { NextResponse } from 'next/server';
2  import { createPayPalOrder } from '@lib/paypal/createPayPalOrder';
3
4  export async function POST(req: Request) {
5    try {
6      const body = await req.json();
7      const resBody = await createPayPalOrder(body);
8      return NextResponse.json(resBody);
9    } catch (err: unknown) {
10     console.error('Create Order Error', err);
11     return NextResponse.json({ error: err instanceof Error ? err.message : String(err) }, { status: 500 });
12   }
13 }

```

13b) Intent endpoint (calls createPayPalOrder directly)

New file:

- `src/app/api/paypal/tx-ledger/intent/route.ts`

Purpose:

- Create ledger row first.
- Call `createPayPalOrder(...)` directly (no self-referential HTTP fetch).
- Return `orderToken + paypalOrderId`.

Important ownership note:

- Store only `orderToken` in browser state.
- Return `paypalOrderId` only so `createOrder()` can hand it to the PayPal SDK.
- After capture succeeds, the client should act only as a status consumer. Durable completion belongs to webhook + server runner.

```

ts
1  import { NextResponse } from 'next/server';
2  import { randomUUID } from 'crypto';
3  import { paypalTxLedger } from '@lib/prisma/shop/paypal/paypalTxLedger';
4  import { PAYPAL_LEDGER_STATUS } from '@lib/paypal/txLedger/status';
5  import { createPayPalOrder } from '@lib/paypal/createPayPalOrder';
6

```

```

7   export async function POST(req: Request) {
8       try {
9           const body = await req.json();
10          const {
11              cart,
12              customer,
13              country,
14              country_iso_3,
15              initialCurrency,
16              delivery_address,
17              userId,
18              djangoOrderIntentUuid,
19              djangoOrderIntentOrderId,
20              djangoOrderIntentPayload,
21              djangoOrderIntentVerifyPayload,
22          } = body;
23
24          if (!Array.isArray(cart) || cart.length === 0) {
25              return NextResponse.json({ error: 'Cart required' }, { status: 400 });
26          }
27          if (!customer?.name || !customer?.email) {
28              return NextResponse.json({ error: 'Customer required' }, { status: 400 });
29          }
30          if (!delivery_address) {
31              return NextResponse.json({ error: 'Delivery address required' }, { status: 400 });
32          }
33
34          const orderToken = randomUUID();
35
36          await paypalTxLedger.paypalIntent.create({
37              data: {
38                  orderToken,
39                  status: PAYPAL_LEDGER_STATUS.INTENT_CREATING,
40                  customerName: customer.name,
41                  customerEmail: customer.email,
42                  userId: userId ?? null,
43                  djangoOrderIntentUuid: djangoOrderIntentUuid ?? null,
44                  djangoOrderIntentOrderId: djangoOrderIntentOrderId ?? null,
45                  djangoOrderIntentPayload: djangoOrderIntentPayload ?? null,
46                  djangoOrderIntentVerifyPayload: djangoOrderIntentVerifyPayload ?? null,
47                  countryIso2: country ?? null,
48                  countryIso3: country_iso_3 ?? null,
49                  initialCurrency: initialCurrency ?? null,
50                  cartSnapshot: cart,
51                  shippingSnapshot: delivery_address,
52              },
53          });
54
55          let paypalOrder: any;
56          try {
57              paypalOrder = await createPayPalOrder({
58                  cart,
59                  customer,
60                  country,
61                  country_iso_3,
62                  initialCurrency,
63                  delivery_address,
64              });
65          } catch (createErr) {
66              await paypalTxLedger.paypalIntent.update({
67                  where: { orderToken },
68                  data: {
69                      status: PAYPAL_LEDGER_STATUS.ERROR,
70                      lastErrorCode: 'CREATE_ORDER_FAILED',
71                      lastErrorMessage: createErr instanceof Error ? createErr.message : String(createErr),
72                  },
73              });
74              return NextResponse.json({ error: 'Create order failed' }, { status: 500 });
75          }

```

```

76
77     if (!paypalOrder?.id) {
78         await paypalTxLedger.paypalIntent.update({
79             where: { orderToken },
80             data: {
81                 status: PAYPAL_LEDGER_STATUS.ERROR,
82                 lastErrorCode: 'MISSING_PAYPAL_ORDER_ID',
83                 lastErrorMessage: 'createPayPalOrder returned no id',
84             },
85         });
86         return NextResponse.json({ error: 'Missing PayPal order ID' }, { status: 500 });
87     }
88
89     await paypalTxLedger.paypalIntent.update({
90         where: { orderToken },
91         data: {
92             paypalOrderId: paypalOrder.id,
93             status: PAYPAL_LEDGER_STATUS.INTENT_CREATED,
94         },
95     });
96
97     return NextResponse.json({
98         orderToken,
99         paypalOrderId: paypalOrder.id,
100     });
101 } catch (err) {
102     return NextResponse.json({ error: 'Intent failed' }, { status: 500 });
103 }
104 }

```

14) PayPalCheckoutChildren Integration (replace createOrderAction path)

Current file:

- `src/components/UI/Shop/Checkout/Paypal/PayPalCheckoutChildren.tsx`

Replace the `createOrder` callback to call `/next-api/paypal/tx-ledger/intent`.

Important behavior:

- The intent route returns both `orderToken` and `paypalOrderId`.
- Store only `orderToken` in `paypalIntentStore`.
- Return `paypalOrderId` directly to the PayPal SDK.
- Do not persist `paypalOrderId` in your own client app state after `createOrder()` returns.

Required payload sources (codebase-verified)

- `cart` from `useCartStore`
- `customer` + `delivery_address` from `useShopCheckoutStore`
- `country_iso2`, `country_iso3`, `currency` from `ServerOrderDetailsContext`

- `djangoOrderIntentUuid`, `djangoOrderIntentOrderId`, and `djangoOrderIntentVerifyPayload` from `djangoOrderIntentStore`

Template (drop-in replacement shape)

ts

```
1 import { useDjangoOrderIntentStore } from '@stores/shop_stores/checkoutStore/djangoOrderIntentStore';
2 import { usePayPalIntentStore } from '@stores/shop_stores/checkoutStore/paypalIntentStore';
3
4 // inside component
5 const {
6   djangoOrderIntentUuid,
7   djangoOrderIntentOrderId,
8   djangoOrderIntentPayload,
9   djangoOrderIntentVerifyPayload,
10 } = useDjangoOrderIntentStore();
11 const setIntent = usePayPalIntentStore((s) => s.setIntent);
12
13 const createOrder = useCallback(async (): Promise<string> => {
14   const payload = {
15     cart,
16     customer: {
17       name: `${first_name} ${last_name}`,
18       email: email ?? 'john@example.com',
19     },
20     country: country_iso2 ?? 'US',
21     country_iso_3: country_iso3 ?? 'USA',
22     initialCurrency: currency ?? 'USD',
23     delivery_address,
24     djangoOrderIntentUuid,
25     djangoOrderIntentOrderId,
26     djangoOrderIntentPayload,
27     djangoOrderIntentVerifyPayload,
28   };
29
30   const res = await fetch('/next-api/paypal/tx-ledger/intent', {
31     method: 'POST',
32     headers: { 'Content-Type': 'application/json' },
33     body: JSON.stringify(payload),
34   });
35
36   if (!res.ok) {
37     const text = await res.text();
38     throw new Error(`PayPal intent failed: ${text}`);
39   }
40
41   const data = await res.json();
42   if (!data?.orderToken || !data?.paypalOrderId) {
43     throw new Error('Invalid intent response');
44   }
45
46   setIntent({ orderToken: data.orderToken });
47   return data.paypalOrderId as string;
48 }, [
49   cart,
50   first_name,
51   last_name,
52   email,
53   country_iso2,
54   country_iso3,
55   currency,
56   delivery_address,
57   djangoOrderIntentUuid,
58   djangoOrderIntentOrderId,
```

```
59     djangoOrderIntentPayload,  
60     djangoOrderIntentVerifyPayload,  
61     setIntent,  
62   });
```

Why this replaces `createOrderAction` in PayPal button path:

- Prevents duplicate order creation paths.
- Ensures ledger row exists before PayPal approve/capture.

15) Authorize Route Update (ledger write)

Current file:

- `src/app/api/paypal/orders/authorize/route.ts`

Add `orderToken` in request body and persist:

- `authorizePayload`
- `paypalAuthorizationId`
- `status`

ts

```
1   import { NextResponse } from 'next/server';  
2   import { OrdersController } from '@paypal/paypal-server-sdk';  
3   import { paypalClient } from '@lib/paymentClients/paypalClient';  
4   import { paypalTxLedger } from '@lib/prisma/shop/paypal/paypalTxLedger';  
5   import { PAYPAL_LEDGER_STATUS } from '@lib/paypal/txLedger/status';  
6  
7   const orders = new OrdersController(paypalClient);  
8  
9   export async function POST(req: Request) {  
10    const { orderId, orderToken } = await req.json();  
11    if (!orderId || !orderToken) {  
12      return NextResponse.json({ error: 'orderId and orderToken required' }, { status: 400 });  
13    }  
14  
15    try {  
16      const { body: resBody } = await orders.authorizeOrder({  
17        id: orderId,  
18        prefer: 'return=representation',  
19      });  
20  
21      const authorizationId =  
22        resBody?.purchaseUnits?.[0]?.payments?.authorizations?.[0]?.id ??  
23        resBody?.purchase_units?.[0]?.payments?.authorizations?.[0]?.id;  
24  
25      await paypalTxLedger.paypalIntent.update({  
26        where: { orderToken },  
27        data: {  
28          paypalAuthorizationId: authorizationId ?? null,  
29          authorizePayload: resBody as unknown as object,  
30          status: PAYPAL_LEDGER_STATUS.AUTHORIZED,  
31        },  
32      });  
33  
34      return NextResponse.json(resBody);
```

```

35   } catch (err: unknown) {
36     await paypalTxLedger.paypalIntent.update({
37       where: { orderToken },
38       data: {
39         status: PAYPAL_LEDGER_STATUS.ERROR,
40         lastErrorCode: 'AUTHORIZE_FAILED',
41         lastErrorMessage: err instanceof Error ? err.message : String(err),
42       },
43     }).catch(() => undefined);
44
45     return NextResponse.json({ error: 'Failed to authorize' }, { status: 500 });
46   }
47 }

```

16) Capture Route Update (ledger write)

Current file:

- `src/app/api/paypal/orders/capture/route.ts`

This section needs the same hardening mindset as Section 15. The minimal "call PayPal, then write the DB row" shape is not enough on its own, because PayPal capture can succeed while ledger persistence fails afterward.

Required outcomes:

- accept `orderToken` plus `authorizationId`
- persist `capturePayload`
- persist `status`
- reject invalid state transitions
- avoid double-capturing on retries
- return structured client-safe errors

Recommended defaults:

- use `PayPal-Request-Id` so repeated capture attempts are idempotent at PayPal's side
- if the row is already `captured` and `capturePayload` exists, return the stored payload
- if the row is `error`, only retry with the same deterministic `PayPal-Request-Id`
- if capture succeeds but ledger persistence fails, return a specific persistence error instead of pretending the whole capture failed normally

ts

```

1   import { randomUUID } from 'crypto';
2   import { NextResponse } from 'next/server';
3   import { PaymentsController, type CapturedPayment } from '@paypal/paypal-server-sdk';
4   import { paypalClient } from '@lib/paymentClients/paypalClient';
5   import { paypalTxLedger } from '@lib/prisma/shop/paypal/paypalTxLedger';
6   import { PAYPAL_LEDGER_STATUS } from '@lib/paypal/txLedger/status';
7   import { createPayPalRouteResponders } from '@lib/paypal/txLedger/routeResponses';
8
9   const payments = new PaymentsController(paypalClient);
10
11  export async function POST(req: Request) {
12    const requestId = randomUUID();

```

```

13 let orderToken: string | undefined;
14
15 const { error: routeError } = createPayPalRouteResponders({
16   requestId,
17   getOrderToken: () => orderToken,
18 });
19
20 const validationError = (code: string, message: string, status = 400) =>
21   routeError({
22     status,
23     code,
24     stage: 'validate_request',
25     message,
26   });
27
28 const fail = async ({
29   30   stage,
30   c
31   o
32   d
33   e
34   ,
35   36   err,
36   m
37   e
38   s
39   s
40   a
41   g
42   e
43   ,
44   45   persistToLedger = true,
45   s
46   t
47   a
48   t
49   u
50   s
51   =
52   5
53   0
54   0
55   ,
56   57   code: string;
57   }
58   :
59   {
60   61   message: string;
61   s
62   t
63   a
64   g
65   e
66   :
67   s
68   t
69   r
70   i
71   n
72   g
73   ;
74   75   status?: number;
75   e
76   r
77   r
78   :
79   u
80   n
81   k

```

```

n
o
w
n
;
41 42    }
    p
    e
    r
    s
    i
    s
    t
    T
    o
    L
    e
    d
    g
    e
    > {
    e
    r
    ?
    :
    b
    o
    o
    l
    e
    a
    n
    ;

43    console.error(`[paypal.capture.${stage}]`, {
44        requestId,
45        orderToken,
46        code,
47        error: err instanceof Error ? err.message : String(err),
48        stack: err instanceof Error ? err.stack : undefined,
49    });
50
51    if (persistToLedger && orderToken) {
52        await paypalTxLedger.paypalIntent.update({
53            where: { orderToken },
54            data: {
55                status: PAYPAL_LEDGER_STATUS.ERROR,
56                lastErrorCode: code,
57                lastErrorMessage: err instanceof Error ? err.message : String(err),
58            },
59        }).catch(() => undefined);
60    }
61
62    return routeError({
63        status,
64        code,
65        stage,
66        message,
67    });
68 };
69
70 const persistCapturedResult = async (payload: CapturedPayment) => {
71     await paypalTxLedger.paypalIntent.update({
72         where: { orderToken },
73         data: {
74             status: PAYPAL_LEDGER_STATUS.CAPTURED,
75             capturePayload: JSON.parse(JSON.stringify(payload)),
76             lastErrorCode: null,
77             lastErrorMessage: null,
78         },
79     });
80 };

```



```

81
82 try {
83     const { authorizationId, orderToken: requestOrderToken } = await req.json();
84     orderToken = requestOrderToken;
85
86     if (!authorizationId || !orderToken) {
87         return validationError(
88             'INVALID_CAPTURE_REQUEST',
89             'authorizationId and orderToken are required',
90         );
91     }
92
93     const intent = await paypalTxLedger.paypalIntent.findUnique({
94         where: { orderToken },
95     });
96
97     if (!intent) {
98         return routeError({
99             status: 404,
100             code: 'INTENT_NOT_FOUND',
101             stage: 'load_intent',
102             message: 'Intent not found',
103         });
104     }
105
106     if (intent.paypalAuthorizationId && intent.paypalAuthorizationId !== authorizationId) {
107         return routeError({
108             status: 409,
109             code: 'AUTHORIZATION_MISMATCH',
110             stage: 'validate_intent',
111             message: 'Authorization mismatch',
112         });
113     }
114
115     if (intent.status === PAYPAL_LEDGER_STATUS.CAPTURED && intent.capturePayload) {
116         return Response.json(intent.capturePayload);
117     }
118
119     if (
120         intent.status === PAYPAL_LEDGER_STATUS.RECEIPT_UPLOADED ||
121         intent.status === PAYPAL_LEDGER_STATUS.PAYMENT_SAVED ||
122         intent.status === PAYPAL_LEDGER_STATUS.COMPLETED ||
123         intent.status === PAYPAL_LEDGER_STATUS.REFUNDED
124     ) {
125         return routeError({
126             status: 409,
127             code: 'CAPTURE_STATE_CONFLICT',
128             stage: 'validate_intent',
129             message: `Cannot capture intent from status "${intent.status}"`,
130         });
131     }
132
133     if (
134         intent.status !== PAYPAL_LEDGER_STATUS.AUTHORIZED &&
135         intent.status !== PAYPAL_LEDGER_STATUS.ERROR
136     ) {
137         return routeError({
138             status: 409,
139             code: 'CAPTURE_STATE_INVALID',
140             stage: 'validate_intent',
141             message: `Unexpected intent status "${intent.status}" before capture`,
142         });
143     }
144
145     const { result } = await payments.captureAuthorizedPayment({
146         authorizationId,
147         paypalRequestId: `capture:${orderToken}`,
148         prefer: 'return=representation',
149         body: { finalCapture: true },

```

```

150     });
151
152     try {
153         await persistCapturedResult(result);
154     } catch (persistErr) {
155         return fail({
156             code: 'CAPTURE_PERSIST_FAILED',
157             stage: 'persist_capture_payload',
158             message: 'PayPal capture succeeded but ledger persistence failed',
159             err: persistErr,
160             persistToLedger: false,
161         });
162     }
163
164     return Response.json(result);
165 } catch (err: unknown) {
166     return fail({
167         code: 'CAPTURE_FAILED',
168         stage: 'capture_payment',
169         message: 'Failed to capture PayPal authorization',
170         err,
171     });
172 }
173 }

```

Notes:

- Capture recovery differs slightly from authorize: the safe default here is provider-side idempotency via a stable `paypalRequestId`, plus returning cached `capturePayload` when the ledger already has it.
- If you retry capture with a new request id after an ambiguous failure, you are accepting the risk of a second provider-side capture attempt.

17) Approve Hook Update (usePayPalTXApproveCallback)

Current file:

- `src/lib/hooks/shopHooks/checkout/usePayPalTXApproveCallback.ts`

Required changes

1. Fix parsing bug:

- remove `JSON.parse(await res.json())`

2. Pass `orderToken` from `paypalIntentStore` to authorize/capture routes.

3. Do not call client post-processing functions in this flow.

4. Redirect to the confirmation route, where polling becomes the read path.

5. Treat the client as a status viewer after capture.

- Capture still happens through your API route.
- Durable completion happens later through webhook + server runner.
- Local client state is convenience only, never the source of truth for completion.
- The canonical recovery handle is `orderToken` in the URL, not long-lived client store state.

Template fragment (core changes)

```
ts
1  import { usePayPalIntentStore } from '@stores/shop_stores/checkoutStore/paypalIntentStore';
2
3  const orderToken = usePayPalIntentStore((s) => s.orderToken);
4
5  // authorize
6  const authRes = await fetch('/next-api/paypal/orders/authorize', {
7    method: 'POST',
8    headers: { 'Content-Type': 'application/json' },
9    body: JSON.stringify({ orderID: data.orderID, orderToken }),
10  });
11  if (!authRes.ok) {
12    throw new Error(await readRouteErrorMessage(authRes, 'Authorization failed'));
13  }
14  const authData = (await authRes.json()) as OrderResponseBody;
15
16  // capture
17  const capRes = await fetch('/next-api/paypal/orders/capture', {
18    method: 'POST',
19    headers: { 'Content-Type': 'application/json' },
20    body: JSON.stringify({ authorizationId, orderToken }),
21  });
22  if (!capRes.ok) {
23    throw new Error(await readRouteErrorMessage(capRes, 'Capture failed'));
24  }
25  const capturedOrder = (await capRes.json()) as OrdersCapture;
26
27  // start UI, then move the browser into confirmation/status-view mode
28  startProcessingUI(capturedOrder.id);
29  router.push(`/shop/checkout/confirmation/${encodeURIComponent(orderToken)}`);
```

Minimal shared error reader:

```
ts
1  async function readRouteErrorMessage(res: Response, fallback: string) {
2    try {
3      const payload = (await res.json()) as {
4        error?: { message?: string; code?: string; stage?: string; requestId?: string };
5      };
6
7      if (!payload?.error) return fallback;
8
9      const { code, stage, requestId, message } = payload.error;
10     return `${stage ?? 'unknown_stage'} ${code ?? 'UNKNOWN_ERROR'} (${requestId ?? 'no_request_id'}): ${message ?? fallback}`;
11   } catch {
12     return fallback;
13   }
14 }
```

Important cleanup in this hook

When ledger flow is enabled:

- remove runtime usage of:
 - `usePost_PaymentProcessors`
 - `useServerActionWithState(...)` wrappers for receipt/payment/merchize
 - `buildServerPostProcessingFunc(...)`

Keep:

- processing UI state setters
- error toasts
- authorize/capture orchestration

Keep the hook thin:

- read the structured route error
- initialize the processing UI
- push the user to the confirmation route carrying `orderToken`
- let the confirmation page poll ledger truth

Do not make this hook responsible for receipt upload, backend save, or fulfillment retries.

If your shop runs on a dedicated domain with path normalization, use your shop-specific router abstraction here instead of raw `next/navigation` routing. The redirect target stays `/shop/checkout/confirmation/[orderToken]`, but the push helper should normalize it correctly for both the parent domain and the dedicated shop domain.

18) PayPal TX Payment Status Endpoint

New file:

- `src/app/api/paypal/tx-ledger/payments/[orderToken]/status/route.ts`

Returns ledger truth for polling.

This endpoint should be intentionally small and browser-safe:

- use `orderToken` as the lookup key
- do not treat client state as authoritative
- do not expose `paypalOrderId` unless you have a concrete browser-side need for it
- return only the fields the confirmation UI actually needs

```
ts
1  import { NextResponse } from 'next/server';
2  import { paypalTxLedger } from '@lib/prisma/shop/paypal/paypalTxLedger';
3
4  type PageProps = {
5    params: Promise<{ orderToken: string }>;
```

```

6   };
7
8   export async function GET(_req: Request, { params }: PageProps) {
9     const { orderToken } = await params;
10
11    const row = await paypalTxLedger.paypalIntent.findUnique({ where: { orderToken } });
12    if (!row) {
13      return NextResponse.json({ error: 'Not found' }, { status: 404 });
14    }
15
16    return NextResponse.json({
17      orderToken: row.orderToken,
18      status: row.status,
19      lastEventType: row.lastEventType,
20      receiptLink: row.receiptLink,
21      receiptFile: row.receiptFile,
22      djangoOrderIntentUuid: row.djangoOrderIntentUuid,
23      djangoOrderIntentOrderId: row.djangoOrderIntentOrderId,
24      djangoPaymentSaveCustomId: row.djangoPaymentSaveCustomId,
25      processingCompletedAt: row.processingCompletedAt,
26      error: row.lastErrorMessage
27      ? { code: row.lastErrorCode, message: row.lastErrorMessage }
28      : null,
29    });
30  }

```

Optional hardening:

- If you later want to keep support correlation available, add a separate admin endpoint that can expose `paypalOrderId` server-side instead of returning it to the browser here.

19) Polling + UI Mapping (Processing Modal / Confirmation Page)

This section fills a missing piece from earlier versions: concrete polling behavior.

Polling rules

- Start polling on the confirmation page after redirect from `usePayPalTXApproveCallback`.
- Poll every 2-3 seconds.
- Stop polling when status is one of:
 - `completed`
 - `error`
 - `refunded`
- Also stop on unmount.
- Prefer `orderToken` from the URL if available. Treat the store as a short-lived bridge, not the recovery source.
- If the UI thinks it is "processing" but there is no `orderToken`, clear the stale local processing state instead of hanging indefinitely.

Why polling is still the default here:

- This guide assumes polling first because it is the simplest correct rollout for your current architecture.
- The confirmation client only needs occasional status changes, not a high-frequency stream.
- A 2-3 second interval is operationally cheap at low to moderate concurrency and keeps failure recovery simple.
- If you later outgrow polling, upgrade to SSE while keeping the payment-status endpoint as fallback.
- Do not choose gRPC for browser status updates in this flow. Browser support is not a good fit for a simple one-way status channel, and SSE is operationally simpler for confirmation-page style updates.

Example polling helper (confirmation-page local)

```
ts

1  const startLedgerPolling = (orderToken?: string) => {
2    if (!orderToken) return;
3
4    let cancelled = false;
5    const interval = 2500;
6
7    const tick = async () => {
8      if (cancelled) return;
9      try {
10       const res = await fetch(
11         `/next-api/paypal/tx-ledger/payments/${encodeURIComponent(orderToken)}/status`,
12         { cache: 'no-store' },
13       );
14       if (!res.ok) return;
15       const data = await res.json();
16       applyLedgerStatusToUI(data);
17
18       if (['completed', 'error', 'refunded'].includes(data.status)) {
19         cancelled = true;
20         return;
21       }
22     } catch {
23       // swallow + retry next tick
24     }
25     if (!cancelled) setTimeout(tick, interval);
26   };
27
28   void tick();
29
30   return () => {
31     cancelled = true;
32   };
33   };
```

Practical rule:

- the modal can use persisted UI state for continuity
- the confirmation page must reconcile that state against the ledger on mount
- ledger truth wins after refresh, reconnect, or browser restart

UI mapping adapter template

File:

- `src/lib/paypal/txLedger/mapLedgerToProcessingState.ts`

ts

```
1  type LedgerStatusResponse = {
2    status: string;
3    receiptLink?: string | null;
4    receiptFile?: string | null;
5    error?: { code?: string | null; message?: string | null } | null;
6  };
7
8  export function mapLedgerToProcessingState(data: LedgerStatusResponse) {
9    const status = data.status;
10
11    if (status === 'completed') {
12      return {
13        flowStatus: 'completed' as const,
14        steps: {
15          receiptUpload: 'success',
16          paymentSave: 'success',
17          orderPush: 'success',
18        },
19      };
20    }
21
22    if (status === 'payment_saved') {
23      return {
24        flowStatus: 'processing' as const,
25        steps: {
26          receiptUpload: 'success',
27          paymentSave: 'success',
28          orderPush: 'pending',
29        },
30      };
31    }
32
33    if (status === 'receipt_uploaded') {
34      return {
35        flowStatus: 'processing' as const,
36        steps: {
37          receiptUpload: 'success',
38          paymentSave: 'pending',
39          orderPush: 'idle',
40        },
41      };
42    }
43
44    if (status === 'error') {
45      return {
46        flowStatus: 'error' as const,
47        steps: null, // derive best-effort from last known status later if needed
48      };
49    }
50
51    return {
52      flowStatus: 'processing' as const,
53      steps: {
54        receiptUpload: 'pending',
55        paymentSave: 'idle',
56        orderPush: 'idle',
57      },
58    };
59  }
```

Confirmation page integration

Current file:

- `src/app/shop/checkout/confirmation/[orderToken]/page.tsx`

The confirmation page should use `orderToken` as the route param. This keeps the browser-facing identifier aligned with the ledger key and avoids exposing `paypalOrderId` in the URL.

This is the preferred recovery model:

- the path segment is the ledger lookup key
- if the user refreshes, the page can resume from ledger truth without depending on in-memory client state

How orderToken reaches the confirmation page

In the approve hook (`usePayPalTXApproveCallback`), after capture succeeds, redirect with `orderToken` :

```
ts

1  // after successful capture in the approve hook
2  push(`/shop/checkout/confirmation/${encodeURIComponent(orderToken)}`);
```

Confirmation page polling template (add to existing page)

```
tsx

1  'use client';
2
3  import { use } from 'react';
4  import { useEffect, useState } from 'react';
5  import { mapLedgerToProcessingState } from '@lib/paypal/txLedger/mapLedgerToProcessingState';
6
7  export function useLedgerPolling({
8    9    }: {
9      p
10     a
11     r
12     a
13     m
14     s
15     ,
16     11    }
17     p
18     a
19     r
20     a
21     m
22     s
23     :
24     P
25     r
26     o
27     m
28     i
29     s
30     e
31     <
32     {
33     o
```



```

) {
  d
  e
  r
  T
  o
  k
  e
  n
  :
  s
  t
  r
  i
  n
  g
}
>
;

12   const orderToken = use(params).orderToken;
13   const [processingState, setProcessingState] = useState<ReturnType<typeof mapLedgerToProcessingState> | null>
    (null);

14
15   useEffect(() => {
16     if (!orderToken) return;
17
18     let cancelled = false;
19     const interval = 2500;
20
21     const tick = async () => {
22       if (cancelled) return;
23       try {
24         const res = await fetch(
25           `/next-api/paypal/tx-ledger/payments/${encodeURIComponent(orderToken)}/status`,
26           { cache: 'no-store' },
27         );
28         if (!res.ok) return;
29         const data = await res.json();
30         setProcessingState(mapLedgerToProcessingState(data));
31
32         if (['completed', 'error', 'refunded'].includes(data.status)) {
33           cancelled = true;
34           return;
35         }
36       } catch {
37         // swallow, retry next tick
38       }
39       if (!cancelled) setTimeout(tick, interval);
40     };
41
42     void tick();
43     return () => { cancelled = true; };
44   }, [orderToken]);
45
46   return { orderToken, processingState };
47 }

```

Important: Do not clear checkout state immediately on mount

Deferred fix #7 notes that the current confirmation page clears checkout/order state on mount. With ledger polling, **delay clearing** until `processingState.flowStatus === 'completed'` so recovery context survives.

Browser-close / returning user recovery

If the user's browser closes after capture but before `completed`:

1. The ledger row already exists with status `captured` (or later).
2. The webhook will still fire and trigger `runPostProcessing`.
3. If the user returns, the confirmation page URL (bookmarked or in history) still has `orderToken` in the path — polling resumes automatically.
4. If the user lost the URL entirely, admin can look up by `customerEmail` and share status or receipt link.

No separate "check my order" page is required. The webhook guarantees completion regardless of browser state.

20) Webhook Route Upgrade (idempotent + post-processing trigger)

Current file:

- `src/app/api/paypal/webhooks/ledger-transaction-events/route.ts`

Keep your existing signature verification and raw-body handling.

Additions required

1. Delivery tracking by `PaypalWebhookEvent.eventId` plus `processedAt` / `attemptCount`
2. Robust PayPal order id extraction
3. Retry when the ledger row is not yet correlated
4. `runPostProcessing(orderToken)` trigger on `PAYMENT.CAPTURE.COMPLETED`
5. ACK fast; do not let the webhook route become the place where long receipt/backend/fulfillment work blocks the provider response

Why `after()` for post-processing (Next.js 15+)

PayPal expects a response within ~10 seconds. If `runPostProcessing` takes longer (receipt upload, Merchize API, payment save), the webhook times out and PayPal marks the delivery as failed — triggering retries that overlap with your still-running work.

`after()` from `next/server` solves this: the webhook handler responds 200 immediately, and the post-processing callback runs in the same Node.js process after the response is sent. Your VPS runs Node.js directly (not edge/serverless), so `after()` is fully supported.

Tradeoff: since we respond 200 before post-processing finishes, PayPal considers the delivery successful. If the Node process crashes mid-`after()`, PayPal will not retry. The webhook event stays in `received` status — recover via admin retry (Section 24) or a periodic recovery query:

ts

```
1 // Find webhook events stuck in 'received' for more than 10 minutes
2 const stale = await paypalTxLedger.paypalWebhookEvent.findMany({
3   where: {
4     processingStatus: 'received',
5     createdAt: { lt: new Date(Date.now() - 10 * 60 * 1000) },
6   },
7 });
```

This tradeoff is standard in production webhook systems — ACK fast, process async, recover from your own state.

Robust PayPal order ID extraction helper

PayPal event shapes vary. Use fallback extraction:

ts

```
1 function getWebhookPaypalOrderId(event: any): string | undefined {
2   return (
3     event?.resource?.supplementary_data?.related_ids?.order_id ||
4     event?.resource?.custom_id ||
5     event?.resource?.invoice_id ||
6     undefined
7   );
8 }
```

Note:

- For your flow, primary path is still `supplementary_data.related_ids.order_id`.
- `custom_id` / `invoice_id` are reasonable fallbacks if you later set them explicitly during create-order.
- Do not use unrelated fields as fake fallbacks just to avoid `undefined`. If correlation fails, record that explicitly and let the webhook retry or recovery path handle it.

Default idempotent webhook core template (merge into your route)

ts

```
1 import { after } from 'next/server';
2 import { paypalTxLedger } from '@lib/prisma/shop/paypal/paypalTxLedger';
3 import { PAYPAL_LEDGER_STATUS } from '@lib/paypal/txLedger/status';
4 import { runPostProcessing } from '@lib/paypal/txLedger/runPostProcessing';
5
6 // after verification + parsed `event`
7 const eventId = event.id;
8 const eventType = event.event_type;
9 const paypalOrderId = getWebhookPaypalOrderId(event);
10
11 if (!eventId) return new Response('Missing event id', { status: 400 });
12
13 let webhookEvent = await paypalTxLedger.paypalWebhookEvent.findUnique({
14   where: { eventId },
15 });
16
17 if (!webhookEvent) {
18   webhookEvent = await paypalTxLedger.paypalWebhookEvent.create({
```

```

19     data: {
20         eventId,
21         eventType,
22         paypalOrderId: paypalOrderId ?? null,
23         payload: event,
24         processingStatus: 'received',
25     },
26 });
27 }
28
29 if (webhookEvent.processedAt) {
30     return new Response('OK', { status: 200 });
31 }
32
33 if (!paypalOrderId) {
34     await paypalTxLedger.paypalWebhookEvent.update({
35         where: { eventId },
36         data: {
37             processingStatus: 'ignored',
38             processedAt: new Date(),
39             lastAttemptAt: new Date(),
40             attemptCount: { increment: 1 },
41             lastErrorMessage: null,
42         },
43     });
44
45     return new Response('OK', { status: 200 });
46 }
47
48 const row = await paypalTxLedger.paypalIntent.findUnique({ where: { paypalOrderId } });
49 if (!row) {
50     await paypalTxLedger.paypalWebhookEvent.update({
51         where: { eventId },
52         data: {
53             processingStatus: 'awaiting_ledger_row',
54             lastAttemptAt: new Date(),
55             attemptCount: { increment: 1 },
56             lastErrorMessage: 'Ledger row not correlated yet',
57         },
58     });
59
60     return new Response('Retry later', { status: 500 });
61 }
62
63 if (row.status === PAYPAL_LEDGER_STATUS.COMPLETED) {
64     await paypalTxLedger.paypalWebhookEvent.update({
65         where: { eventId },
66         data: {
67             processingStatus: 'processed',
68             processedAt: new Date(),
69             lastAttemptAt: new Date(),
70             attemptCount: { increment: 1 },
71             lastErrorMessage: null,
72         },
73     });
74
75     return new Response('OK', { status: 200 }); // never rerun completion
76 }
77
78 try {
79     switch (eventType) {
80         case 'PAYMENT.AUTHORIZATION.CREATED':
81             await paypalTxLedger.paypalIntent.update({
82                 where: { paypalOrderId },
83                 data: { status: PAYPAL_LEDGER_STATUS.AUTHORIZED, lastEventType: eventType },
84             });
85             break;
86
87         case 'PAYMENT.CAPTURE.PENDING':

```

```

88     await paypalTxLedger.paypalIntent.update({
89       where: { paypalOrderId },
90       data: { status: PAYPAL_LEDGER_STATUS.PENDING, lastEventType: eventType },
91     });
92     break;
93
94   case 'PAYMENT.CAPTURE.DENIED':
95     await paypalTxLedger.paypalIntent.update({
96       where: { paypalOrderId },
97       data: {
98         status: PAYPAL_LEDGER_STATUS.ERROR,
99         lastEventType: eventType,
100        lastErrorCode: 'CAPTURE_DENIED',
101        lastErrorMessage: 'PayPal capture denied',
102      },
103    });
104    break;
105
106   case 'PAYMENT.CAPTURE.REFUNDED':
107     await paypalTxLedger.paypalIntent.update({
108       where: { paypalOrderId },
109       data: { status: PAYPAL_LEDGER_STATUS.REFUNDED, lastEventType: eventType },
110     });
111     break;
112
113   case 'PAYMENT.CAPTURE.COMPLETED':
114     if (
115       ![
116         PAYPAL_LEDGER_STATUS.RECEIPT_UPLOADED,
117         PAYPAL_LEDGER_STATUS.PAYMENT_SAVED,
118         PAYPAL_LEDGER_STATUS.COMPLETED,
119       ].includes(row.status)
120     ) {
121       await paypalTxLedger.paypalIntent.update({
122         where: { paypalOrderId },
123         data: { status: PAYPAL_LEDGER_STATUS.CAPTURED, lastEventType: eventType },
124       });
125     }
126
127     // Schedule post-processing to run AFTER the response is sent to PayPal.
128     // This prevents webhook timeouts under load. The DB-backed lease in
129     // runPostProcessing prevents duplicate work if a retry overlaps.
130     after(async () => {
131       try {
132         await runPostProcessing(row.orderToken);
133
134         await paypalTxLedger.paypalWebhookEvent.update({
135           where: { eventId },
136           data: {
137             processingStatus: 'processed',
138             processedAt: new Date(),
139             lastAttemptAt: new Date(),
140             attemptCount: { increment: 1 },
141             lastErrorMessage: null,
142           },
143         });
144       } catch (err) {
145         const message = err instanceof Error ? err.message : String(err);
146
147         await paypalTxLedger.paypalWebhookEvent.update({
148           where: { eventId },
149           data: {
150             processingStatus: 'failed',
151             lastAttemptAt: new Date(),
152             attemptCount: { increment: 1 },
153             lastErrorMessage: message,
154           },
155         });
156       }
157     });

```

```

157     await paypalTxLedger.paypalIntent.update({
158       where: { paypalOrderId },
159       data: {
160         lastErrorCode: 'POST_PROCESSING_FAILED',
161         lastErrorMessage: message,
162       },
163     });
164   }
165 });
166
167 // Respond 200 immediately. Post-processing continues via after().
168 // If the process crashes before after() completes, the webhook event
169 // stays in 'received' status – recover via admin retry (Section 24).
170 return new Response('OK', { status: 200 });
171 }
172
173 await paypalTxLedger.paypalWebhookEvent.update({
174   where: { eventId },
175   data: {
176     processingStatus: 'processed',
177     processedAt: new Date(),
178     lastAttemptAt: new Date(),
179     attemptCount: { increment: 1 },
180     lastErrorMessage: null,
181   },
182 });
183 } catch (err) {
184   const message = err instanceof Error ? err.message : String(err);
185
186   await paypalTxLedger.paypalWebhookEvent.update({
187     where: { eventId },
188     data: {
189       processingStatus: 'failed',
190       lastAttemptAt: new Date(),
191       attemptCount: { increment: 1 },
192       lastErrorMessage: message,
193     },
194   });
195
196   await paypalTxLedger.paypalIntent.update({
197     where: { paypalOrderId },
198     data: {
199       lastErrorCode: 'POST_PROCESSING_FAILED',
200       lastErrorMessage: message,
201     },
202   });
203
204   return new Response('Webhook processing failed', { status: 500 });
205 }
206
207 return new Response('OK', { status: 200 });

```

Operational rule:

- the webhook is the durable completion trigger
- the browser is not part of that trust boundary
- if the client disappears entirely after capture, the webhook path must still be sufficient to complete the order

21) Server Post-Processing Runner (ledger-driven)

This is the core server-only replacement for client post-processing.

New file:

- `src/lib/paypal/txLedger/runPostProcessing.ts`

Important:

- This section is ledger-focused.
- Provider-specific fulfillment logic is delegated (see Section 22).
- The runner must be restart-safe and retry-safe.
- The runner must be able to finish the order from ledger state alone, without any browser-only context.

Important: server-only crypto

The existing `encrypt` helper currently lives in `@/stores/shop_stores/cartStore`, which is a `'use client'` file and depends on a public env var. Do **not** reuse that helper from webhook/server code.

Keep two helpers:

- client-side checkout encryption can stay on its current path until you refactor the OTP flow
- webhook/post-processing must use a server-only helper backed by `SHOP_SERVER_ACTIONS_CRYPT0_SECRET`

```
ts

1  // src/lib/utils/shop/checkout/serverPostProcessingCrypto.ts
2  import 'server-only';
3  import CryptoJS from 'crypto-js';
4
5  function getServerActionSecret() {
6    const secret = process.env.SHOP_SERVER_ACTIONS_CRYPT0_SECRET;
7    if (!secret) {
8      throw new Error('SHOP_SERVER_ACTIONS_CRYPT0_SECRET is not configured');
9    }
10   return secret;
11 }
12
13 export function encryptForPostProcessingServerAction(text: string): string {
14   return CryptoJS.AES.encrypt(text, getServerActionSecret()).toString();
15 }
```

Update the decrypt side of `savePaymentReceiptToCloud`, `savePaymentDataToBackend`, and fulfillment server actions to use the same server-only secret. Do not import this module into client code.

```
ts

1  import { randomUUID } from 'crypto';
2  import { encryptForPostProcessingServerAction } from '@lib/utils/shop/checkout/serverPostProcessingCrypto';
3  import { paypalTxLedger } from '@lib/prisma/shop/paypal/paypalTxLedger';
4  import { PAYPAL_LEDGER_STATUS } from '@lib/paypal/txLedger/status';
5  import { savePaymentReceiptToCloud } from
6    '@actions/shop/paypal/processAndUploadCompletedTx/savePaymentReceiptToCloud';
7  import { savePaymentDataToBackend } from
8    '@actions/shop/paypal/processAndUploadCompletedTx/savePaymentDataToBackend';
9  import { sendMerchizeFulfillmentOrder } from '@lib/paypal/txLedger/sendMerchizeFulfillmentOrder';
```

```

8
9  const POST_PROCESSING_LEASE_MS = 5 * 60_000;
10
11 export async function runPostProcessing(orderToken: string) {
12     const now = new Date();
13     const leaseExpiresAt = new Date(now.getTime() + POST_PROCESSING_LEASE_MS);
14     const lockId = randomUUID();
15
16     const lease = await paypalTxLedger.paypalIntent.updateMany({
17         where: {
18             orderToken,
19             processingCompletedAt: null,
20             OR: [
21                 { postProcessingLockExpiresAt: null },
22                 { postProcessingLockExpiresAt: { lt: now } },
23             ],
24         },
25         data: {
26             postProcessingLockId: lockId,
27             postProcessingLockedAt: now,
28             postProcessingLockExpiresAt: leaseExpiresAt,
29         },
30     });
31
32     if (lease.count === 0) return;
33
34     const row = await paypalTxLedger.paypalIntent.findUnique({ where: { orderToken } });
35     if (!row) throw new Error('Ledger row not found');
36
37     if (row.postProcessingLockId !== lockId) return;
38
39     if (row.processingCompletedAt || row.status === PAYPAL_LEDGER_STATUS.COMPLETED) {
40         await paypalTxLedger.paypalIntent.updateMany({
41             where: { orderToken, postProcessingLockId: lockId },
42             data: {
43                 postProcessingLockId: null,
44                 postProcessingLockedAt: null,
45                 postProcessingLockExpiresAt: null,
46             },
47         });
48         return;
49     }
50
51     try {
52         const authData = row.authorizePayload as any;
53         const finalCapturedOrder = row.capturePayload as any;
54         if (!authData || !finalCapturedOrder) {
55             throw new Error('Missing authorize/capture payload in ledger');
56         }
57
58         const customer = { name: row.customerName, email: row.customerEmail };
59         const ORD_string = row.djangoOrderIntentOrderId ?? row.orderToken;
60
61         // Step 1: Receipt
62         if (!row.receiptLink || !row.receiptFile) {
63             const receiptPayload = encryptForPostProcessingServerAction(JSON.stringify({ authData, customer,
64 ORD_string }));
65             const receiptRes = await savePaymentReceiptToCloud(receiptPayload);
66             if (!receiptRes.success) {
67                 throw new Error(receiptRes.message ?? 'Receipt upload failed');
68             }
69
70             await paypalTxLedger.paypalIntent.updateMany({
71                 where: { orderToken, postProcessingLockId: lockId },
72                 data: {
73                     status: PAYPAL_LEDGER_STATUS.RECEIPT_UPLOADED,
74                     receiptLink: receiptRes.pdfReceiptLink,
75                     receiptFile: receiptRes.receiptFileName,
76                     lastErrorCode: null,

```



```

76     lastErrorMessage: null,
77     },
78     });
79
80     row.receiptLink = receiptRes.pdfReceiptLink;
81     row.receiptFile = receiptRes.receiptFileName;
82 }
83
84 // Step 2: Payment save
85 if (!row.djangoPaymentSaveCustomId) {
86     const savePayload = encryptForPostProcessingServerAction(
87         JSON.stringify({
88             authData,
89             finalCapturedOrder,
90             capturedOrderPaypalID: row.paypalOrderId,
91             customer,
92             delivery_address: row.shippingSnapshot,
93             userId: row.userId,
94             ORD_string,
95             pdfReceiptLink: row.receiptLink,
96             receiptFileName: row.receiptFile,
97         }),
98     );
99
100     const paymentSave = await savePaymentDataToBackend(savePayload);
101     if (!paymentSave.ok) {
102         throw new Error(paymentSave.error?.message ?? 'Payment save failed');
103     }
104
105     await paypalTxLedger.paypalIntent.updateMany({
106         where: { orderToken, postProcessingLockId: lockId },
107         data: {
108             status: PAYPAL_LEDGER_STATUS.PAYMENT_SAVED,
109             djangoPaymentSaveCustomId: paymentSave.data?.custom_id ?? null,
110             lastErrorCode: null,
111             lastErrorMessage: null,
112         },
113     });
114
115     row.djangoPaymentSaveCustomId = paymentSave.data?.custom_id ?? null;
116 }
117
118 if (!row.djangoPaymentSaveCustomId) {
119     throw new Error('Missing Django payment save custom_id before fulfillment push');
120 }
121
122 // Step 3: Merchize fulfillment
123 await sendMerchizeFulfillmentOrder({
124     cartSnapshot: row.cartSnapshot,
125     djangoPaymentSaveCustomId: row.djangoPaymentSaveCustomId,
126     identifier: 'codexchristi-shop',
127     currency: row.initialCurrency ?? 'USD',
128     customerName: row.customerName,
129     fulfillmentAddress: row.shippingSnapshot,
130 });
131
132 await paypalTxLedger.paypalIntent.updateMany({
133     where: { orderToken, postProcessingLockId: lockId },
134     data: {
135         status: PAYPAL_LEDGER_STATUS.COMPLETED,
136         processingCompletedAt: new Date(),
137         postProcessingLockId: null,
138         postProcessingLockedAt: null,
139         postProcessingLockExpiresAt: null,
140         lastErrorCode: null,
141         lastErrorMessage: null,
142     },
143 });
144 } catch (err) {

```

```

145     const message = err instanceof Error ? err.message : String(err);
146
147     await paypalTxLedger.paypalIntent.updateMany({
148       where: { orderToken, postProcessingLockId: lockId },
149       data: {
150         lastErrorCode: 'POST_PROCESSING_FAILED',
151         lastErrorMessage: message,
152         postProcessingLockId: null,
153         postProcessingLockedAt: null,
154         postProcessingLockExpiresAt: null,
155       },
156     });
157
158     throw err;
159   }
160 }

```

Operational note:

- If receipt upload, payment save, or fulfillment can exceed the lease duration, renew `postProcessingLockExpiresAt` between steps.
- Pass `orderToken` as the downstream idempotency key wherever the target API supports it.
- Treat the ledger row as the single source of truth for what still needs to run. Do not rebuild required inputs from client stores at this stage.
- Each step should be safely skippable if its success artifact already exists in the ledger row.

22) Merchize Fulfillment Helper

This section isolates Merchize fulfillment payload construction from the main ledger runner.

Current file:

- `src/lib/paypal/txLedger/sendMerchizeFulfillmentOrder.ts`

Ledger-facing helper shape

```

ts
1  // src/lib/paypal/txLedger/sendMerchizeFulfillmentOrder.ts
2  import { sendMerchizeOrderDetailsToBackend } from
   '@actions/shop/checkout/createMerchizeOrder/sendMerchizeOrderDetailsToBackend';
3  import { encryptForPostProcessingServerAction } from '@lib/utils/shop/checkout/serverPostProcessingCrypto';
4
5  export type MerchizeFulfillmentOrderArgs = {
6    cartSnapshot: CartVariant[];
7    djangoPaymentSaveCustomId: string;
8    identifier: string;
9    currency: string;
10   customerName: string;
11   fulfillmentAddress: PaymentSavingActionProps['delivery_address'];
12 };
13
14 export async function sendMerchizeFulfillmentOrder(args: MerchizeFulfillmentOrderArgs) {
15   const requestPayload = buildFulfillmentPayload(
16     args,

```

```

17     mapCartToProcessingItems(args.cartSnapshot, args.currency),
18   );
19
20   const response = await sendMerchizeOrderDetailsToBackend(
21     encryptForPostProcessingServerAction(
22       JSON.stringify({
23         djangoPaymentSaveCustomId: args.djangoPaymentSaveCustomId,
24         payload: requestPayload,
25       }),
26     ),
27   );
28
29   if (!response.ok) throw new Error(response.error?.message ?? 'Fulfillment push failed');
30   return { ...response, requestPayload };
31 }

```

Why this matters:

- The Django payment save custom ID is the fulfillment process path identifier.
- The fulfillment request payload is saved back to the ledger as `merchizeFulfillmentRequestPayload`.
- The fulfillment response payload is saved back to the ledger as `merchizeFulfillmentResponsePayload`.

23) Admin Auth (header secret)

Decision default:

- Header-only auth for now (cookie/session auth is an optional upgrade — see Section 31).
- No cookie/session logic in these templates.

New file:

- `src/lib/utils/adminAuth.ts`

ts

```

1  export function assertAdminSecret(req: Request) {
2    const provided = req.headers.get('x-admin-secret');
3    const expected = process.env.ADMIN_SECRET_KEY;
4
5    if (!expected) {
6      throw new Error('ADMIN_SECRET_KEY is not configured');
7    }
8
9    if (!provided || provided !== expected) {
10     const err = new Error('Unauthorized');
11     (err as Error & { status?: number }).status = 401;
12     throw err;
13   }
14 }

```

24) Admin Ledger UI + Retry Actions (Search, Filter, Retry)

New files:

- `src/app/admin/shop/paypal-merchize-ledger/page.tsx`
- `src/app/admin/shop/paypal-merchize-ledger/actions.ts`
- `src/app/admin/shop/paypal-merchize-ledger/AdminLedgerTable.tsx`

Goals for the admin UI

- Search by:
 - `orderToken`
 - `paypalOrderId`
 - `customerEmail`
 - `djangoOrderIntentUuid`
 - `djangoOrderIntentOrderId`
 - `djangoPaymentSaveCustomId`
- Filter by:
 - `status`
 - date window (optional simple version)
- Retry:
 - full post-processing (`receipt -> payment save -> fulfillment`)
 - later (optional) step-specific retries
- Inspect:
 - last error code/message
 - retry count
 - created/updated timestamps

Server actions template

```
ts
1 // src/app/admin/shop/paypal-merchize-ledger/actions.ts
2 'use server';
3
4 import { paypalTxLedger } from '@lib/prisma/shop/paypal/paypalTxLedger';
5 import { runPostProcessing } from '@lib/paypal/txLedger/runPostProcessing';
6
7 export async function retryPostProcessing(orderToken: string) {
8   const row = await paypalTxLedger.paypalIntent.findUnique({ where: { orderToken } });
9   if (!row) throw new Error('Ledger row not found');
10
11   await paypalTxLedger.paypalIntent.update({
12     where: { orderToken },
13     data: { retryCount: { increment: 1 } },
```

```

14   });
15
16   await runPostProcessing(orderToken);
17 }

```

Query helpers for search/filter (server-side)

This is the missing practical part: the admin page should not fetch everything unfiltered.

ts

```

1  // add to src/app/admin/shop/paypal-merchize-ledger/actions.ts (or a local server helper file)
2  'use server';
3
4  import { paypalTxLedger } from '@lib/prisma/shop/paypal/paypalTxLedger';
5
6  type ListLedgerArgs = {
7    q?: string;
8    status?: string;
9    page?: number;
10   pageSize?: number;
11 };
12
13 export async function listLedgerRows({
14   15   status,
15   q
16   ,
17   pageSize = 25,
18   p
19   a
20   g
21   e
22   =
23   1
24   ,
25 }
26 :
27 L
28 i
29 s
30 t
31 L
32 e
33 d
34 g
35 e
36 r
37 A
38 r
39 g
40 s
41 ) {
42   const safePage = Math.max(1, page);
43   const safePageSize = Math.min(100, Math.max(10, pageSize));
44   const skip = (safePage - 1) * safePageSize;
45
46   const where = {
47     ...(status ? { status } : {}),
48     ...(q
49       ? {
50         OR: [
51           { orderToken: { contains: q, mode: 'insensitive' as const } },
52           { paypalOrderId: { contains: q, mode: 'insensitive' as const } },
53           { customerEmail: { contains: q, mode: 'insensitive' as const } },

```

```

31     { djangoOrderIntentUuid: { contains: q, mode: 'insensitive' as const } },
32     { djangoOrderIntentOrderId: { contains: q, mode: 'insensitive' as const } },
33     { djangoPaymentSaveCustomId: { contains: q, mode: 'insensitive' as const } },
34   ],
35   }
36   : {}),
37 };
38
39 const [rows, total] = await Promise.all([
40   paypalTxLedger.paypalIntent.findMany({
41     where,
42     orderBy: { createdAt: 'desc' },
43     skip,
44     take: safePageSize,
45   }),
46   paypalTxLedger.paypalIntent.count({ where }),
47 ]);
48
49 return {
50   rows,
51   total,
52   page: safePage,
53   pageSize: safePageSize,
54   totalPages: Math.max(1, Math.ceil(total / safePageSize)),
55 };
56 }

```

Admin page route template (server component shell)

tsx

```

1  // src/app/admin/shop/paypal-merchize-ledger/page.tsx
2  import AdminLedgerTable from './AdminLedgerTable';
3  import { listLedgerRows } from './actions';
4
5  type PageProps = {
6    searchParams?: Promise<{
7      q?: string;
8      status?: string;
9      page?: string;
10    }>;
11  };
12
13  export default async function PayPalLedgerAdminPage({ searchParams }: PageProps) {
14    const params = (await searchParams) ?? {};
15    const q = params.q?.trim() || '';
16    const status = params.status?.trim() || '';
17    const page = Number(params.page || '1');
18
19    const data = await listLedgerRows({
20      q: q || undefined,
21      status: status || undefined,
22      page: Number.isFinite(page) ? page : 1,
23      pageSize: 25,
24    });
25
26    return (
27      <main className='p-6 space-y-6'>
28        <h1 className='text-2xl font-semibold'>PayPal TX Ledger</h1>
29      <
30      h
31      e
32      a
33      d

```



```

    {
      data
      rows
    }
  }
  38    totalPages={data.totalPages}
    page
    =
    {
      data
      .
      page
    }
  40    41    status={status}
    query
    =
    {
      query
    }
  42  /
  >
43  <
  /
  memo
  i
  n
  >
44  );
45  }

```

Admin table UI template (search + filter + retry)

tsx

```

1  // src/app/admin/shop/paypal-merchize-ledger/AdminLedgerTable.tsx
2  'use client';
3
4  import { useMemo, useTransition } from 'react';
5  import { useRouter, useSearchParams } from 'next/navigation';
6  import { retryPostProcessing } from './actions';
7
8  type LedgerRow = {
9    id: string;
10   orderToken: string;
11   paypalOrderId: string | null;
12   customerEmail: string;

```



```

13     djangoOrderIntentUuid: string | null;
14     djangoOrderIntentOrderId: string | null;
15     djangoPaymentSaveCustomId: string | null;
16     status: string;
17     retryCount: number;
18     lastErrorCode: string | null;
19     lastErrorMessage: string | null;
20     createdAt: Date;
21     updatedAt: Date;
22 };
23
24 type Props = {
25     rows: LedgerRow[];
26     total: number;
27     page: number;
28     totalPages: number;
29     q: string;
30     status: string;
31 };
32
33 const STATUS_OPTIONS = [
34     '',
35     'intent_creating',
36     'intent_created',
37     'authorized',
38     'captured',
39     'receipt_uploaded',
40     'payment_saved',
41     'pending',
42     'completed',
43     'refunded',
44     'error',
45 ];
46
47 export default function AdminLedgerTable({
48     49     total,
49     r
50     o
51     w
52     s
53     ,
54     51     totalPages,
55     p
56     a
57     g
58     e
59     ,
60     53     status,
61     q
62     ,
63     }
64     :
65     P
66     r
67     o
68     p
69     s
70 ) {
71     const router = useRouter();
72     const searchParams = useSearchParams();
73     const [isPending, startTransition] = useTransition();
74
75     const setQuery = (next: { q?: string; status?: string; page?: number }) => {
76         const params = new URLSearchParams(searchParams?.toString() ?? '');
77         if (next.q !== undefined) {
78             next.q ? params.set('q', next.q) : params.delete('q');
79         }
80         if (next.status !== undefined) {
81             next.status ? params.set('status', next.status) : params.delete('status');

```

```

66     }
67     if (next.page !== undefined) {
68         next.page > 1 ? params.set('page', String(next.page)) : params.delete('page');
69     }
70     startTransition(() => {
71         router.replace(`/admin/shop/paypal-merchize-ledger?${params.toString()}`);
72     });
73 };
74
75 const canPrev = page > 1;
76 const canNext = page < totalPages;
77
78 const summary = useMemo(
79     () => `Showing ${rows.length} of ${total} total records`,
80     [rows.length, total],
81 );
82
83 return (
84     <section className='space-y-4'>
85         88         e.preventDefault();
86         87
87         c         o
88         l         n
89         a         S
90         s         u
91         s         b
92         N         m
93         a         i
94         m         t
95         e         =
96         =         {
97         ,         (
98         g         e
99         r         )
100        i         =
101        d         >
102        g
103        a
104        p
105        -
106        3
107        m
108        d
109        :
110        g
111        r
112        i
113        d
114        -
115        c
116        o
117        l
118        s
119        {
120        [
121        1
122        f
123        r
124        -
125        2
126        2
127        0
128        p
129        x

```



```
b
u
t
t
o
n
>
114 115 <div className='text-sm opacity-80'>{summary}</div>
116 117 <div className='overflow-x-auto border rounded'>
118 119 <thead className='bg-black/5'>
<
t
a
b
l
e
c
l
a
s
s
N
a
m
e
=
,
w
-
f
u
l
l
t
e
x
t
-
s
m
,
>
120 121 <th className='text-left p-2'>Status</th>
<
t
r
>
122 123 <th className='text-left p-2'>Order Token</th>
<
t
h
c
l
a
s
s
N
a
m
e
=
,
t
e
x
t
-
l
e
```



```

page +
1 } })
|
|
i
s
p
e
n
d
i
n
g
}

190 191 Next
192 193 </div>
194 195 <
196 196 /
s
e
c
t
i
o
n
>
);
}

```

Optional improvements (safe to add later)

- Step-specific retry buttons:
 - `Retry Receipt Only`
 - `Retry Payment Save + Fulfillment`
 - `Retry Fulfillment Only`
- JSON payload inspector drawer for authorize/capture payloads
- CSV export for filtered results
- date-range filtering

Note:

- If you want route-level auth later, add a protected wrapper route or middleware.
- Can also be restricted by reverse proxy/private network.

25) Sandbox/Dev Fallback Trigger

Why:

- PayPal sandbox webhook delivery/testing can be inconsistent depending on environment and simulator behavior.
- You still need to validate the ledger post-processing path.

New file:

- `src/app/api/dev/paypal/replay-post-processing/route.ts`

```
ts

1  import { NextResponse } from 'next/server';
2  import { assertAdminSecret } from '@lib/utils/adminAuth';
3  import { runPostProcessing } from '@lib/paypal/txLedger/runPostProcessing';
4
5  export async function POST(req: Request) {
6    if (process.env.NODE_ENV === 'production') {
7      return NextResponse.json({ error: 'Not allowed in production' }, { status: 403 });
8    }
9
10   try {
11     assertAdminSecret(req);
12   } catch (err) {
13     return NextResponse.json({ error: 'Unauthorized' }, { status: 401 });
14   }
15
16   const { orderToken } = await req.json();
17   if (!orderToken) {
18     return NextResponse.json({ error: 'Missing orderToken' }, { status: 400 });
19   }
20
21   await runPostProcessing(orderToken);
22   return NextResponse.json({ ok: true });
23 }
```

This endpoint is a **dev-only tool**, not part of production payment correctness.

26) Rollout and Migration Order (safe deployment)

This is the missing operational sequence from earlier drafts.

Step 1: Database foundation

1. Add schema file.
2. Run `prisma format` + `prisma validate`.
3. Run ledger migrations.
4. Generate ledger Prisma client.

Step 2: Write-only ledger instrumentation (no behavior change yet)

1. Add ledger client wrapper.
2. Add status constants/types.
3. Add intent endpoint.
4. Update `PayPalCheckoutChildren` to create intents and store `orderToken`.
5. Update authorize/capture routes to write ledger payloads and statuses.

At this point:

- Existing checkout behavior should still work.
- Authorize/capture still happen synchronously through your existing API routes.
- The client is not the durable owner of completion after capture.
- Client post-processing remains disabled as currently coded.

Step 3: Read path + UI sync

1. Add order status endpoint.
2. Add polling and UI mapping.
3. Verify processing UI updates from ledger.

Step 4: Durable completion path

1. Add webhook idempotency and status transitions.
2. Add post-processing runner.
3. Add fulfillment adapter isolation.
4. Trigger post-processing from webhook.

Step 5: Admin and recovery tooling

1. Add admin auth helper.
2. Add admin page + retry action.
3. Add dev fallback trigger endpoint.

Step 6: Cleanup

1. Remove runtime client post-processing code path from `usePayPalTXApproveCallback`.
2. Keep server actions (receipt/payment save/fulfillment) because webhook runner still uses them.

Rollback safety

- If webhook/post-processing breaks:
 - Keep ledger writes in place.
 - Disable webhook trigger path.
 - Use admin retry or dev fallback for recovery while patching.
-

27) Package Scripts (no extra Prisma config file)

Add scripts that explicitly target the ledger schema file.

```
json
1  {
2    "scripts": {
3      "prisma:merchize:generate": "prisma generate --config prisma.config.ts",
4      "prisma:paypal:validate": "prisma validate --schema prisma/shop/paypal/paypalTXLedger.schema.prisma",
5      "prisma:paypal:format": "prisma format --schema prisma/shop/paypal/paypalTXLedger.schema.prisma",
6      "prisma:paypal:generate": "prisma generate --schema prisma/shop/paypal/paypalTXLedger.schema.prisma",
7      "prisma:paypal:migrate:dev": "prisma migrate dev --schema prisma/shop/paypal/paypalTXLedger.schema.prisma",
8      "prisma:paypal:migrate:deploy": "prisma migrate deploy --schema
prisma/shop/paypal/paypalTXLedger.schema.prisma"
9    }
10 }
```

Why this is safe:

- Merchize flow remains config-driven.
 - PayPal ledger flow remains schema-path-driven.
 - No cross-target confusion.
-

28) Testing Matrix (local/sandbox/live)

Local dev (no reliable webhook)

- Create intent from checkout.
- Authorize/capture routes write ledger payloads.
- Poll order status endpoint.
- Use dev fallback endpoint to trigger `runPostProcessing(orderToken)`.
- Verify:
 - receipt link/file saved
 - Django payment save custom ID saved
 - final status `completed`

Sandbox webhook testing

- Enable webhook verification as needed.
- Confirm webhook rows dedupe by `eventId`.
- Confirm `completed` rows are not reprocessed.

Production readiness checks

- Webhook verification ON.
 - `ADMIN_SECRET_KEY` set.
 - Dev fallback route blocked in production.
 - Monitoring/logs for:
 - `CREATE_ORDER_FAILED`
 - `AUTHORIZE_FAILED`
 - `CAPTURE_FAILED`
 - `POST_PROCESSING_FAILED`
-

29) Deferred Known Issues in Current Code (tracked)

These are already tracked in:

- `PAYPAL_TX_LEDGER_DEFERRED_FIXES.md`

Highlights (do not lose track of these during ledger work):

- `JSON.parse(await res.json())` bug in `usePayPalTXApproveCallback`
 - create-order route catch returning 200 JSON on failure
 - partially disabled processing modal flow
 - OTP signature secret exposed client-side (`NEXT_PUBLIC_*`) - separate security issue
-

30) Decision Defaults

These defaults are implemented in this guide's templates. Items marked **(deferred)** are intentionally kept simple for now with a known upgrade path.

1. `ORD_string` source

- Keep dual references.
- Internal primary key: `orderToken`
- External/business fallback: `djangoOrderIntentOrderId ?? orderToken`

- `djangoOrderIntentOrderId` comes from the verified Django order-intent OTP flow, not from a client-generated order string.

2. Webhook + sandbox testing mode

- Add dev-only fallback endpoint for `runPostProcessing(orderToken)`
- Disabled in production

3. Admin auth transport **(deferred)**

- Header-only auth via `x-admin-secret`
- No cookie/session logic in these templates

4. Status vocabulary **(deferred)**

- String statuses + centralized constants file
- Prisma enum conversion deferred

5. Capture/authorize operation ownership

- Keep existing authorize/capture API routes as ownership point
- Webhook is for confirmation, retries, and durable completion
- Capture writes the immediate `captured` state; webhook + server runner own everything after that

6. Post-processing execution location

- Server-only (webhook + runner)
- Client does polling + UI only
- Client state is convenience only; ledger truth wins after refresh/reconnect

7. Error handling policy

- Persist `lastErrorCode` + `lastErrorMessage` on ledger row
- Prefer structured route errors with stable `code`, `stage`, and `requestId`

8. Idempotency policy

- Track webhook delivery by `eventId`, but only mark it processed after business work succeeds
- Use a DB-backed post-processing lease so only one worker owns receipt/payment/fulfillment at a time
- Never rerun post-processing when ledger row already `completed`

9. Receipt timing

- Generate/upload receipt after `PAYMENT.CAPTURE.COMPLETED` only

10. Data retention baseline

- Keep raw authorize/capture payloads in JSON columns for reconciliation
-

31) Optional Overrides (only if rejecting defaults)

Use this section only if you intentionally want to diverge from Section 30.

1. `ORD_string` override
 - Use `orderToken` only and stop storing the Django order-intent order ID.
 2. Sandbox override
 - Disable dev fallback endpoint and require webhook-only testing.
 3. Admin auth override
 - Replace header-only auth with cookie/session auth.
 4. Status override
 - Convert statuses to Prisma enum immediately.
 5. Provider coupling override
 - Inline Merchize calls directly in `runPostProcessing` (not recommended).
-

32) Future: Real-Time Notifications at Scale (SSE + Pub/Sub, not gRPC)

This section does not need to be implemented now. The polling approach in Section 19 is correct and sufficient. Read this when you are ready to replace polling with push-based real-time updates.

Decision default for this codebase:

- Polling first.
- SSE second when polling volume becomes wasteful.
- Keep the PayPal TX payment-status endpoint as the source of truth either way.
- Do not use gRPC for browser-facing payment-status delivery. If you ever introduce gRPC, reserve it for internal service-to-service boundaries, not confirmation-page client updates.

Why polling is fine for now

Polling the payment-status endpoint every 2–3 seconds works well when:

- You have fewer than ~500 concurrent users waiting on the confirmation page.
- Post-processing typically completes in under 30 seconds.
- Your VPS can absorb the request volume ($500 \text{ users} \times 1 \text{ req}/2.5\text{s} = 200 \text{ req/s}$ — manageable).

When polling stops being enough

At 5,000+ concurrent waiters, polling generates thousands of status requests per second. Most return "still processing" — wasted work. Server-Sent Events (SSE) invert this: the server pushes a single notification when the status actually changes.

Why NOT to use in-memory EventEmitter

A common first attempt at SSE:

```
ts

1  // DO NOT use this in production
2  import { EventEmitter } from 'events';
3  const emitter = new EventEmitter();
4
5  // Webhook writes to DB, then:
6  emitter.emit(`paypal-tx-payment-status:${orderToken}`, { status: 'completed' });
7
8  // SSE endpoint listens:
9  emitter.on(`paypal-tx-payment-status:${orderToken}`, (data) => {
10    controller.enqueue(`data: ${JSON.stringify(data)}\n\n`);
11  });
```

This breaks because:

1. **Single-process only.** `EventEmitter` is in-memory. If you run PM2 cluster mode, multiple Node workers, or blue/green deploys, the webhook may hit worker A while the SSE connection is on worker B. Worker B never sees the event.
2. **No persistence.** If the user reconnects (browser tab refresh, network hiccup), the event is gone. The client must fall back to polling anyway.
3. **Memory leaks.** Each SSE connection registers a listener. If you forget to clean up on disconnect, you leak memory proportional to connection count.

What to use instead: Redis Pub/Sub or Postgres LISTEN/NOTIFY

Option A: Redis Pub/Sub (recommended for >1,000 concurrent waiters)

Redis Pub/Sub works across all Node processes and servers. Every worker subscribes; the webhook worker publishes.

```
text

1  npm install ioredis
```

Publisher (inside `runPostProcessing` or `after()` callback, after status update):

```
ts

1  import Redis from 'ioredis';
2
3  const redisPub = new Redis(process.env.REDIS_URL!);
4
5  // After updating ledger status to 'completed':
```

```

6    await redisPub.publish(
7      `paypal-tx-payment-status:${orderToken}`,
8      JSON.stringify({ status: 'completed', completedAt: new Date().toISOString() }),
9    );

```

Subscriber (SSE endpoint):

```

ts

1  import Redis from 'ioredis';
2
3  export async function GET(req: Request) {
4    const url = new URL(req.url);
5    const orderToken = url.searchParams.get('orderToken');
6    if (!orderToken) return new Response('Missing orderToken', { status: 400 });
7
8    const redisSub = new Redis(process.env.REDIS_URL!);
9    const channel = `paypal-tx-payment-status:${orderToken}`;
10
11   const stream = new ReadableStream({
12     start(controller) {
13       // Send heartbeat every 30s to keep the connection alive
14       const heartbeat = setInterval(() => {
15         controller.enqueue(`: heartbeat\n\n`);
16       }, 30_000);
17
18       redisSub.subscribe(channel);
19
20       redisSub.on('message', (_ch, message) => {
21         controller.enqueue(`data: ${message}\n\n`);
22         // Clean up after delivering the terminal event
23         clearInterval(heartbeat);
24         redisSub.unsubscribe(channel);
25         redisSub.quit();
26         controller.close();
27       });
28
29       // Clean up if client disconnects
30       req.signal.addEventListener('abort', () => {
31         clearInterval(heartbeat);
32         redisSub.unsubscribe(channel);
33         redisSub.quit();
34         controller.close();
35       });
36     },
37   });
38
39   return new Response(stream, {
40     headers: {
41       'Content-Type': 'text/event-stream',
42       'Cache-Control': 'no-cache',
43       Connection: 'keep-alive',
44       'X-Accel-Buffering': 'no', // disable Nginx buffering
45     },
46   });
47 }

```

Option B: Postgres LISTEN/NOTIFY (no extra infrastructure)

If you do not want to add Redis, Postgres has built-in pub/sub via `LISTEN` / `NOTIFY`. This works across all Node processes connected to the same database.

Publisher (raw SQL after ledger status update):

ts

```
1 await prisma.$executeRawUnsafe(  
2   `NOTIFY order_status, '${JSON.stringify({ orderToken, status: 'completed' })}'`,  
3 );
```

Subscriber (SSE endpoint, using a raw pg connection):

ts

```
1 import { Client } from 'pg';  
2  
3 export async function GET(req: Request) {  
4   const url = new URL(req.url);  
5   const orderToken = url.searchParams.get('orderToken');  
6   if (!orderToken) return new Response('Missing orderToken', { status: 400 });  
7  
8   const pgClient = new Client(process.env.DATABASE_URL!);  
9   await pgClient.connect();  
10  await pgClient.query('LISTEN order_status');  
11  
12  const stream = new ReadableStream({  
13    start(controller) {  
14      const heartbeat = setInterval(() => {  
15        controller.enqueue(': heartbeat\n\n');  
16      }, 30_000);  
17  
18      pgClient.on('notification', (msg) => {  
19        if (!msg.payload) return;  
20        const data = JSON.parse(msg.payload);  
21        if (data.orderToken !== orderToken) return;  
22  
23        controller.enqueue(`data: ${msg.payload}\n\n`);  
24        clearInterval(heartbeat);  
25        pgClient.end();  
26        controller.close();  
27      });  
28  
29      req.signal.addEventListener('abort', () => {  
30        clearInterval(heartbeat);  
31        pgClient.end();  
32        controller.close();  
33      });  
34    },  
35  });  
36  
37  return new Response(stream, {  
38    headers: {  
39      'Content-Type': 'text/event-stream',  
40      'Cache-Control': 'no-cache',  
41      'Connection': 'keep-alive',  
42      'X-Accel-Buffering': 'no',  
43    },  
44  });  
45 }
```

Postgres LISTEN/NOTIFY limitations:

- Payload max 8,000 bytes (fine for status updates, not for large data).
- Each SSE connection holds a dedicated pg connection. At 1,000+ concurrent, you will hit your Postgres `max_connections` limit. Use Redis Pub/Sub instead at that scale.
- Notifications are fire-and-forget — if the listener connects after the NOTIFY, it misses the event. Always check current DB state on SSE connect (same as Redis).

Choosing between the two

Factor	Redis Pub/Sub	Postgres LISTEN/NOTIFY
Extra infrastructure	Yes (Redis server)	No (your existing Postgres)
Max concurrent SSE connections	10,000+ (limited by Node memory)	~200–500 (limited by pg connections)
Your VPS already runs Redis?	Use this	—
You want zero new dependencies	—	Use this

Keep polling as fallback

Regardless of which push mechanism you choose, **keep the payment-status polling endpoint**. SSE connections can drop due to proxies, network changes, or browser throttling (background tabs). The client should:

1. Open SSE connection on mount.
2. If SSE delivers the terminal status, done.
3. If SSE disconnects or times out, fall back to polling.
4. On reconnect, check current DB state first (don't wait for a new pub/sub event).

VPS proxy configuration (Nginx/Caddy)

Reverse proxies buffer responses by default, which breaks SSE. Add these headers:

Nginx:

```
nginx
1  location /next-api/paypal/tx-ledger/payments/<orderToken>/status/stream {
2      proxy_pass http://localhost:3000;
3      proxy_http_version 1.1;
4      proxy_set_header Connection '';
5      proxy_buffering off;
6      proxy_cache off;
7      proxy_read_timeout 86400s; # keep SSE alive for up to 24h
8  }
```

Caddy (automatic, no config needed — Caddy streams by default when `X-Accel-Buffering: no` is set).